September Attack — Monitor Validation & Incident Correlation

Audience: internal team review. **Status:** findings are complete for Ethereum mainnet; Arbitrum confirmed by direct state-replay (not live pipeline); Filecoin out of scope (not yet monitored by this repo). See Gaps and open items.

---

1. What we set out to answer

Could `sf-monitor`, as designed, have caught the 2026-09-05 exploit (~$176k taken via a red-black tree bug in `OrderStatisticsTreeLib` that let a still-resting order be misclassified as filled and minted unbacked future value)? And if so, at what point — and how confidently — would it have told us to pause?

This is not a retrospective guess. Every claim below is backed by one of: - a direct on-chain state read at the exact historical block (`soak.py`), - a live replay of the real transactions through the real OZ Monitor pipeline and the real trigger scripts, posting to the real Slack webhook, or - a line-by-line cross-check against the independently produced incident report and fund-flow trace.

---

2. The path we took

1. **Validated the integration contract before trusting any of it.** Captured a real OZ Monitor match payload (`MonitorMatch::EVM`) from a manual Sepolia test withdrawal and diffed it against OZ Monitor v1.6.0's own source (`services/filter/filters/evm/filter.rs`, `services/trigger/script/executor.rs`) to confirm field shapes, the `python3 -c <script>` invocation contract, and — critically — that `matched_on_args.events` is a **list of every matching log in the transaction**, not a single event.
2. **Direct state-replay first.** Extended `soak.py` with `--from-block` / `--to-block` and ran the I1 (future-value conservation) and I2 (custody solvency) checks directly against historical chain state across the full attack window, on both Ethereum mainnet and Arbitrum One. This confirmed, independent of any pipeline plumbing: I1 fires in the attacker's first transaction and the breach persists; I2 never fires.
3. **Live pipeline replay on mainnet.** Rewound the monitor's cursor file and temporarily set `max_past_blocks: "unlimited"` so the real OZ Monitor binary re-scanned the real attack blocks and fired the real trigger scripts, posting real alerts to the real Slack channel — proving the whole chain works end to end, not just the predicate logic.
4. **Found and fixed a real bug of our own.** All four trigger scripts (`total_deposits_check.py`, `solvency_check.py`, `fv_conservation_check.py`, `block_unit_price_check.py`) read only `matched_on_args.events[0]`, silently discarding every other matching log in the same transaction. The September transactions routinely carried 3–13 matching events each (one `cleanUpFunds` call settles every maturity a user holds — see §4). Fixed by adding `invariants.all_matched_events()` and iterating every distinct currency/maturity, not just the first. Verified against the 26-case synthetic suite (`test_invariants.py`) — no regressions.
5. **Re-derived the whole trace from the match log, not a second live run.** The original live replay had already captured every match OZ Monitor found (`logs/monitor-matches.log`); re-scanning the chain

again added nothing. Sorted all 33 logged mainnet matches strictly by block number and replayed them through the *fixed* scripts, in order, to get a clean, chronologically faithful Slack trail.
6. **Correlated every alert against the independent incident report and fund-flow diagram** (tx hash to tx hash, not amount-similarity) — see §6.

---

3. The invariant: how much imbalance is "normal"

I1 — future-value conservation, per currency per maturity

```
| lendSupply − borrowSupply |  ≤  pendingOrderAmount × FV_MULTIPLIER      (FV_MULTIPLIER = 2)

lendSupply          FutureValueVault.getTotalLendingSupply(maturity)
borrowSupply        FutureValueVault.getTotalBorrowingSupply(maturity)
pendingOrderAmount  LendingMarketController.getPendingOrderAmount(ccy, maturity)
```

Why there's a tolerance at all, not flat equality: settlement is lazy. Only the partially filled maker side of a match settles synchronously; a fully filled maker settles later, at their own cleanup. Until that cleanup runs, the ledger can look momentarily unbalanced by exactly the amount still waiting to settle — that's what `pendingOrderAmount` measures, and why the tolerance scales with it rather than being a fixed constant.

When `pendingOrderAmount == 0`, the tolerance is zero. Any nonzero gap with nothing pending to explain it is unconditionally a breach — there is no legitimate state that produces it.

Why the multiplier is 2, and what that means for detectability: unit price runs 0–10,000 (10,000 = par). A fill at price `p` mints `amount × 10000 / p` of future value. Working the inequality backward:

```
amount  >  pendingOrderAmount × p / 5000
```

The attack's payload ran at unit price **2** — a near-par-zero price, which is exactly what makes a small amount of real collateral mint a *huge* amount of future value. At that price, the attacker could not have minted more than roughly **1/2,500th of the legitimately-pending settlement pool** without tripping the check. This is the key property: detectability and profitability sit on the same axis. An attacker who raises the price enough to stay under the alarm is, by construction, paying for the position close to honestly — at which point it isn't theft anymore.

Known ceiling (not yet closed): as `p` approaches 5,000, the tolerance band widens enough to permit minting on the order of the entire pending pool. This is a consequence of deriving a future-value tolerance from a present-value counter via a multiplier, rather than tracking pending future value exactly from fill events. Closing this is tracked as future work (`invariant-monitoring-spec.md` §8) — it did not matter for this incident because of how low the attacker's price was, but it is not a general guarantee at every price point.

I2 — custody solvency, per currency

```
IERC20(token).balanceOf(TokenVault)  ≥  TokenVault.getTotalDepositAmount(ccy)
```

No tolerance, no state, one view call. This is the last line of defense — it only fires once money has actually left the vault in excess of what the vault's own books say it holds. It never fired during this attack because the attacker never withdrew more than the (phantom but internally self-consistent) deposit total said was available — the books were lying, but they were lying *consistently*, so solvency-against-the-lie held throughout. This is the textbook case the report also makes: I1 catches it at mint time; I2 is the net underneath in case something slips past I1.

---

4. Alert pseudocode — what each monitor watches and does

All four scripts share the same invocation shape: OZ Monitor spawns `python3 -c <script>` once per matching **transaction**, feeding `{"monitor_match": {"EVM": {...}}, "args": [...]}` on stdin. `matched_on_args.events` is a list — every matching log in that transaction, not just one. (This is the exact thing our own bug got wrong — see §2 step 4.) A single transaction can carry several matches because `cleanUpFunds` settles **every maturity a user holds a position in**, in one call (`FundManagementLogic.sol:819`), emitting one event per maturity — this is also how one transaction can span multiple maturities (see §8).

4.1 Total Deposits — large-withdrawal heuristic (INFO tier)

Watches: `TokenVault.Withdraw`

```
for each distinct currency withdrawn in this transaction:
    token := TokenVault.getTokenAddress(ccy)                # resolved live, not cached
    cur_balance  := ERC20(token).balanceOf(TokenVault) @ this block
    prev_balance := ERC20(token).balanceOf(TokenVault) @ this block - 1
    movement_pct := (cur_balance - prev_balance) / prev_balance × 100

    if movement_pct > -1%:
        continue                                             # not a big enough drop

    net_withdrawn := prev_balance - cur_balance              # real net movement, not any
                                                               # single event's amount —
                                                               # a tx can net several legs
    share_pct := net_withdrawn / prev_balance × 100
    if share_pct < 1%:
        continue

    post_slack_info("Big Withdrawal Detected", currency, net_withdrawn, user, tx)
```

Economic heuristic, not a hard invariant — HIGH severity, never auto-pause, but the fastest human-visible signal that something unusual is moving.

4.2 Custody Solvency — I2 (CRITICAL tier)

Watches: `TokenVault.Withdraw`

```
for each distinct currency withdrawn in this transaction:
    held := ERC20(token).balanceOf(TokenVault) @ this block hash
    owed := TokenVault.getTotalDepositAmount(ccy) @ same block hash
    if held < owed:
        post_slack_critical("Custody solvency breach", currency, held, owed, shortfall, tx)
```

Both reads pinned to the same block **hash** (not a block number or `latest`) — two reads at different heights would manufacture a phantom breach.

4.3 FV Conservation — I1 (CRITICAL tier)

Watches: `FutureValueVault.Transfer` (one monitor per currency's vault address, since `Transfer` carries `maturity` but not `ccy`)

```
for each distinct maturity touched by this transaction:
    lend    := FutureValueVault.getTotalLendingSupply(maturity)   @ block hash
    borrow  := FutureValueVault.getTotalBorrowingSupply(maturity) @ block hash
    pending := LendingMarketController.getPendingOrderAmount(ccy, maturity) @ block hash

    gap     := |lend − borrow|
    if gap > pending × 2:
        unexplained := gap − pending × 2
        post_slack_critical("Future value conservation breach",
                            currency, maturity, lend, borrow, pending, unexplained, tx)
```

4.4 Block Unit Price Fluctuation (INFO tier, no real threshold yet)

Watches: `BlockUnitPriceHistoryUpdated` (emitted by the per-currency `LendingMarket` beacon proxy via `delegatecall` — **not** `LendingMarketController`; this address had to be live-resolved per currency via `CurrencyController.getCurrencies()` + `LendingMarketController.getLendingMarket(ccy)`, confirmed against the Solidity team and a live test tx, because a delegatecall'd library's log always carries the caller's address)

```
for each distinct (currency, maturity) market touched by this transaction:
    history := load_rolling_window(network, currency, maturity)   # last 5 values, JSON file
    history.push(blockUnitPrice); history.trim_to(5)

    change_pct := (history.last − history.first) / history.first × 100
    if change_pct != 0:
        post_slack_info("BlockUnitPrice Fluctuation Detected", currency, maturity,
                        change_pct, history, tx)
```

Ported from the old Defender Autotask as-is: alerts on *any* nonzero change in the window, not a tuned threshold — flagged as still-open in the spec. This script had the same `events[0]`-only bug as the others; fixed in the same pass, though it happened not to change this particular incident's output (every matched transaction here only carried one price-update event).

---

5. Our own detection trail (mainnet, chronological)

Everything below is the corrected, chronologically-replayed Slack output — 9 transactions, block 25,914,035 → 25,914,841 (≈ 13.5 minutes of real block time), zero gaps or out-of-order entries.

Block	Tx (short)	Big Withdrawal	FV Conservation breach
25,914,035	`0x6fdb1869…`	15.79% USDC, 3,000.000002	USDC MAR27 (2027-03-26): lend 2,493,368,287,072 / borrow 11,398,352 / pending 9,999,994
25,914,192	`0xcd615986…`	98.60% WBTC, 0.89899004	WBTC DEC27 (2027-12-31): lend 150,544,190 / borrow 108,108 / pending 997
25,914,405	`0xf3627506…`	99.99% USDC, 16,000.000004	USDC JUN28 (2028-06-30): lend 14,156,940,494 / borrow 0 / pending 999,997
25,914,488	`0x4b3c70d0…`	91.60% ETH, 4.489999999999999004	ETH JUN28 (2028-06-30): lend 24.194158075601368932 / borrow 0 / pending ≈0
25,914,552	`0xeaf30248…`	2.60% JPYC, 113,249.311788	JPYC SEP26 (2026-09-25) (1st read): lend 13,505,776,976,223,829,969,609,425 / borrow 1,740,308,326,945,932,458,659,110 / pending 1,081,111,730,388,373,703,874,270
25,914,571	`0xcf85f506…`	3.04% ETH, 0.012499999999999982	ETH JUN27 (2027-06-25): lend 8.322590915238862741 / borrow 0.000660090457410821 / pending ≈0.0175
25,914,579	`0x8944ba35…`	*(none — mint-only, no Withdraw in this tx)*	JPYC SEP26 (2026-09-25) (2nd read): lend 13,507,801,012,686,476,401,427,186 — gap widening
25,914,652	`0xd87caec6…`	99.87% ETH, 0.398550090368760268	ETH SEP27 (2027-09-24): lend 0.858200842796256794 / borrow 0.000806521596073168 / pending ≈0.5
25,914,679	`0x95c6f289…`	99.84% WBTC, 0.01269964	WBTC JUN28 (2028-06-30) (2nd maturity): lend 0.00744438 / borrow 0 / pending ≈0.0002
25,914,841	`0x51784fa2…`	100.00% JPYC, 4,247,652.823341539735933423	JPYC SEP26 (2026-09-25) (3rd/final read): lend 16,695,064,024,997,937,273,049,290 — final widened gap

No `Custody Solvency` alert anywhere in the trail — I2 never fired, exactly as the mechanism predicts (the vault's accounting and its real balance moved together; only the *future value* ledger was lying).

---

6. Correlation against the incident report and fund-flow trace

The incident report and `QS flows (1).html` independently traced **15 ledger transactions** across three chains, six attacker addresses, five independent clusters. Matching by **tx hash**, not amount similarity:

Ethereum — 9 of 9, full coverage, exact match

Report ID	Tx hash	Report amount	Our monitor amount	Match
T01	`0x6fdb1869659197947facc536823dbb58260d8db0e79949675ae87df2a9b82b9a`	3,000.000002 USDC	3,000.000002 USDC	exact
T02	`0xcd6159860783d8d481cc4bf4dceddc88acdca6bb08f6445525b2904cfd75b1da`	0.89899004 WBTC	0.89899004 WBTC	exact
T03	`0xf3627506a6d7d70ac993722539706e86aa442f69e8200149ba36f11e9eeafcb5`	16,000.000004 USDC	16,000.000004 USDC	exact
T04	`0x4b3c70d0327cec387ec03169d63d85bea964f3bf791fc2f34f76ad663d684c3d`	4.487755 ETH (to builder)	4.489999999999999004 ETH (gross withdrawal)	consistent — gross withdrawal vs. net-of-MEV-fee bid to builder
T05	`0xeaf30248f1f8d1d768b27a1911a91e7682f604fb686eef4bdd8a1c38480ba46d`	113,249.31 JPYC	113,249.311788373703874268 JPYC	exact
T06/T07	`0xcf85f50679393aa291bafc547431e8e75685b05d081be8451dd7d20c0de488e7` and `0xd87caec60a17ad09188590affa2ff1540f2e647b8e42b57d73e661b78f2e6767`	"WETH" (bucketed with T01/T08 in the diagram, not individually numbered)	0.0125 ETH and 0.398550090368760268 ETH	both tx hashes land in our trail; the diagram doesn't disambiguate which hash is T06 vs. T07 beyond the bucket, so neither do we
T08	`0x95c6f289dab7ca06ca84b0d33659308096df01a6a946b6fdf5b1a4ae31864753`	0.01269964 WBTC	0.01269964 WBTC	exact
T12	`0x51784fa25714ca719ba26c1cceec48aa18ee8cb2507602cddb08b41ae57d7796`	4,247,652.82 JPYC	4,247,652.8233415397359 33423 JPYC	exact

Independent double-check, not just tx-hash matching: the report calls out two *specific* standing ETH maturity imbalances by name — "1813881600 = 2027-06-25: 8.32 ETH" and "1845936000 = 2028-06-30: 24.19 ETH." Our own FV Conservation reads, pulled from live contract state rather than copied from the report, landed at **8.322590915238862741 ETH** and **24.194158075601368932 ETH** respectively — the report rounds to 2 decimals, we read the exact on-chain value. This is a second, independent confirmation that didn't come from the same tx-hash lookup as the table above.

The one Ethereum match with no report tx number: `0x8944ba35184832f6e3c83b47fc3837f16abb24f988172179dec64ea8f6d545b4` fired an FV Conservation breach in our trail but isn't individually labeled T01–T15 anywhere in the report or diagram. It's not a false positive — it's a pure phantom-mint transaction (no `Withdraw` in the same tx), one of the three readings that build up the JPYC SEP26 (2026-09-25) standing imbalance the report aggregates into its ~2.6M-FV "poisoned ledger" total rather than calling out by hash. **Nothing in our trail is unexplained; this is the one entry the report folded into an aggregate instead of a named row.**

Arbitrum One — T09, T10, T11: confirmed by direct state-replay, not by a live Slack trail

Report ID	Tx hash	Report amount
T09	`0x9c5485c735e5568b559069a106480abdd1feee27f9bb6f8b23dccebafbb751d1`	7,135.307931 USDC
T10	`0xa55dc996ca45047ace5a44bb72938b2fc06cabf945f8e343e1e3fd5caa9d852b`	0.445256520518573412 WETH
T11	`0xe53c7a02bf63521e80074e6143e470ef22ba97e033a267ebe499a92c489e79ff`	0.00879469 WBTC

`soak.py`'s direct block-range replay against Arbitrum One's real historical state confirmed I1 fires and persists, and I2 never fires, across this window — same mechanism, same result as Ethereum. We deliberately did **not** replay these through the live OZ Monitor pipeline into Slack (Arbitrum's much shorter block time inflates the equivalent block count by roughly 47×, making a live replay ~2.6 hours instead of minutes) — a scope decision, not a detection gap. The underlying predicate was still verified against real chain state for this exact window.

Filecoin — T13, T14, T15: out of scope, not a detection gap in what we tested

Report ID	Tx hash	Report amount
T13	`0x8a49c8096329637e4ddeb8ccda41c96093d519b64e7b61d5bd5c53fdf707dbb3`	28,945.82 USDFC
T14	`0xc0445ae7aaf2d742711a868df47d4b9474cad68bccd2fce9a8de341c35d3f6c0`	8,108.03 WFIL
T15	`0x87c396063d3045706f9bb31e7ce9d045fe184aae1cb4537ffb2a4b9440188ba6`	4,470.06 iFIL seized — a **user's** liquidation loss, not a protocol loss

`sf-monitor` does not configure Filecoin at all yet (pre-existing, documented gap, unrelated to this incident). We make no claim either way about whether I1/I2 would have fired on Filecoin's contracts — that would need the generator and network config extended there first. T15 specifically is a liquidation exploiting the same root-cause bug from a different angle (I3 territory — bounded per-user loss — not I1/I2), and is explicitly excluded from the report's own protocol-loss total.

Net result of the correlation pass

- **9 of 9 Ethereum transactions**: matched by tx hash, amounts exact or independently cross-verified from live state.
- **3 of 3 Arbitrum transactions**: mechanism confirmed against real state for the correct window; not separately replayed into the live alert pipeline (scope choice).
- **3 of 3 Filecoin transactions**: outside current monitoring scope entirely; no claim made.
- **Nothing in our own trail is unaccounted for** — every alert we produced maps to a named report transaction or to an explicitly-aggregated standing imbalance the report itself describes without a per-tx hash.

---

7. Gaps and open items

- `block_unit_price_check.py` had the same `events[0]`-only bug as the other three scripts — found and fixed in this pass (didn't change this incident's output, but was live and wrong for the general case).
- Filecoin is not configured in this monitor at all.
- I3 (bounded per-user loss) is a design sketch only, not implemented — this is the invariant that would have caught T15 (the liquidation) on its own terms, since conservation and solvency both look clean on a liquidation that is unfair rather than unbacked.
- I4 (genesis-value conservation) ships alert-only by design — explicitly not to be armed or trusted on a threshold yet.
- No automated responder exists yet; every alert above requires a human to act on it. The spec's design calls for a minimal, separately-keyed responder process armed only on I1 and I2, holding only `pause` / `unpause` calls — not yet built.
- The known I1 ceiling as unit price approaches 5,000 (§3) is unresolved; it did not matter at this incident's price of 2, but is not a general guarantee.

---

8. How a single transaction touches multiple maturities

Several of the September transactions carried far more matched events than Withdraw/Transfer pairs would suggest at first glance (one tx had 13 `FutureValueVault.Transfer` logs). This is because `FundManagementLogic.cleanUpFunds(ccy, user)` (`FundManagementLogic.sol:819`) settles **every maturity a user currently holds a position in for that currency**, in a single call — looping `getUsedMaturities(ccy, user)` and emitting one settlement event per maturity it touches. `cleanUpAllFunds` does the same across every currency a user holds. This is also exactly the bug behind our own `events[0]` mistake: a script that looks only at the first matched event silently drops every other maturity `cleanUpFunds` settled in that same transaction.

None of the actual September transactions happened to straddle multiple *maturities* for the same user in one call — each stayed within a single currency/maturity pair despite carrying several same-signature events (multiple legs, not multiple maturities) — but the mechanism is real and the fix is general, not specific to this incident.

---

9. At what stage would we have paused

The first unambiguous, pause-worthy signal is the **first CRITICAL `Future value conservation breach` alert** — tx `0x6fdb1869…`, USDC MAR27 (2027-03-26), the very first transaction of the attack. Not the Big Withdrawal that accompanies it (that's an economic heuristic with a benign explanation available — a large legitimate redemption); the breach itself, because I1 encodes a structural fact that has no legitimate reading: a lender claim large enough to dwarf every order currently pending settlement, with no borrower obligation behind it.

Two things make this the actionable moment rather than just an interesting one:

- **It is same-block, same-transaction as the extraction.** The breach and the withdrawal share a tx hash — the phantom value isn't sitting idle, it is being cashed out in the same call that created it. That rules out "maybe this is an indexing artifact" on sight.
- **The invariant is CRITICAL-tier by design, not a tunable threshold.** Per the monitoring spec's own severity model, I1 is "an invariant that cannot break during legitimate operation" — auto-pause eligible, distinct from the HIGH-tier heuristics that require a human judgment call before acting.

Because the root cause lives in a shared order-book library (`OrderStatisticsTreeLib`, inlined into `OrderActionLogic` and `OrderBookLogic`, which every `LendingMarket` proxy delegatecalls), the bug is not specific to the currency that happened to be hit first. The correct response at that first alert is the broad lever — `TokenVault.pause()` protocol-wide — rather than pausing only the one currency that tripped the check, since the first breach alone already proves the underlying defect is live and gives no reason to assume it is confined to USDC.

Acting at that point would have meant the market never stayed open for the second independent attacker cluster to arrive, let alone the three that followed after that — the first transaction's mint-and-extraction can never be undone by any monitor (it is detected and extracted in the same call), but it is also the *only* transaction in the entire incident that has that property. Every other transaction in this trail — across every currency, every cluster, both chains we tested — required the market to still be open and the bug to still be live, both of which a pause at the first alert removes.

Sources: this repo's own live replay (`logs/monitor-matches.log`, re-run through the fixed trigger scripts in corrected chronological order); `invariant-monitoring-spec.md`; `Secured Finance Incident - Technical Interim Report`; `QS flows (1).html` fund-flow trace.